

Step 3 — Deciding What's Worth Remembering, and Writing It Back — Solution

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

All examples below assume Step 2's `state.py`, `store.py`, `graph.py`, and `main.py` already exist and work.

Basic Version

Approach 1 — the direct way, always saving under one key

```
# memory_writer.py (Basic -- no real judgment, saves everything under one key)
from langchain_openai import ChatOpenAI

model = ChatOpenAI(model="gpt-4o-mini")

def judge_message(text: str) -> dict:
    # Basic version: no real "should I save this?" check yet -- just summarizes
    # whatever was said. This is exactly the noisy behavior Intermediate fixes.
    reply = model.invoke([{"role": "user", "content": f"Summarize this in a few w
    return {"worth_remembering": True, "key": "last_message_summary", "value": re
```

Expected output: every single message, including "thanks" and "ok", gets saved -- and each one overwrites `last_message_summary`, so you never actually lose the *previous* fact to a duplicate, but you also never keep more than one fact at a time, and it's usually not the fact you actually wanted kept. This is why Intermediate adds a real judgment step and more than one key.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — a real structured-output judge, with a fixed key list

```
# memory_writer.py
from langchain_openai import ChatOpenAI
from pydantic import BaseModel

ALLOWED_KEYS = ["preferred_name", "stated_preference"]

JUDGE_PROMPT = """Decide if this message contains a fact worth remembering forever
about the user, versus something that only matters today.

Worth remembering (examples): "I'm Sam", "call me Sam, not Samuel",
"I prefer short answers", "always write my emails formally".
NOT worth remembering (examples): "I think I'll have pasta tonight",
"I'm tired today", "thanks", "ok sounds good".

If it IS worth remembering, classify it under exactly one of these keys:
{allowed_keys}

Message: {text}
"""

class MemoryJudgment(BaseModel):
    worth_remembering: bool
    key: str | None = None
    value: str | None = None

judge_model = ChatOpenAI(model="gpt-4o-mini").with_structured_output(MemoryJudgment)

def judge_message(text: str) -> MemoryJudgment:
    prompt = JUDGE_PROMPT.format(allowed_keys=", ".join(ALLOWED_KEYS), text=text)
    return judge_model.invoke([{"role": "user", "content": prompt}])
```

Expected output:

```
>>> judge_message("I think I'll have pasta tonight")
MemoryJudgment(worth_remembering=False, key=None, value=None)
>>> judge_message("I'm Sam, and I prefer being called Sam, not Samuel")
MemoryJudgment(worth_remembering=True, key='preferred_name', value='Sam')
```

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Approach 1 — `write_back_node`, wired into the graph, logging keys not raw values

```
# graph.py (adds write_back_node to Step 2's recall_node + chat_node)
import logging
from functools import partial
from langchain_openai import ChatOpenAI
from langgraph.graph import StateGraph, START, END
from state import ChatState
from store import namespace_for
from memory_writer import judge_message

logger = logging.getLogger(__name__)
model = ChatOpenAI(model="gpt-4o-mini")

def _last_user_text(messages) -> str | None:
    for message in reversed(messages):
        role = getattr(message, "type", None)
        if role == "human":
            return message.content
        if isinstance(message, dict) and message.get("role") == "user":
            return message["content"]
    return None

def write_back_node(state: ChatState, store) -> dict:
    text = _last_user_text(state["messages"])
    if text is None:
        return {}

    judgment = judge_message(text)
    if not judgment.worth_remembering or not judgment.key:
        logger.debug("write_back_node: nothing worth saving this turn")
        return {}

    namespace = namespace_for(state["user_id"])
    store.put(namespace, judgment.key, {"text": judgment.value})
    logger.info("write_back_node: saved fact under key=%s", judgment.key) # not
    return {}
```

```
def build_graph(checkpointer, store):
    from graph_nodes import recall_node, chat_node # Step 2's nodes, unchanged

    builder = StateGraph(ChatState)
    builder.add_node("recall_node", partial(recall_node, store=store))
    builder.add_node("chat_node", chat_node)
    builder.add_node("write_back_node", partial(write_back_node, store=store))
    builder.add_edge(START, "recall_node")
    builder.add_edge("recall_node", "chat_node")
    builder.add_edge("chat_node", "write_back_node")
    builder.add_edge("write_back_node", END)
    return builder.compile(checkpointer=checkpointer)
```

Expected output, run end to end on a brand-new thread:

```
you: I'm Sam. Please call me Sam, not Samuel.
bot: Got it, Sam! How can I help?
[log] write_back_node: saved fact under key=preferred_name

you: I think I'll have pasta tonight.
bot: Sounds good!
[no log line at INFO -- nothing was worth saving]
```

Approach 2 — the overwrite test, proving the dedupe design actually works

```
# test_memory_keeper.py (adds to Step 1 and Step 2's tests)
from store import get_store, namespace_for
from memory_writer import judge_message

def test_forgettable_message_is_not_saved():
    store = get_store()
    before = store.search(namespace_for("dana"), query="anything")
    judgment = judge_message("I think I'll have pasta tonight")
    assert judgment.worth_remembering is False
    after = store.search(namespace_for("dana"), query="anything")
    assert len(before) == len(after) # nothing was added

def test_corrected_name_overwrites_instead_of_duplicating():
    store = get_store()
    namespace = namespace_for("dana")
```

```

first = judge_message("I'm Dana")
store.put(namespace, first.key, {"text": first.value})

second = judge_message("actually, please call me Danielle")
store.put(namespace, second.key, {"text": second.value})

assert first.key == second.key == "preferred_name" # same key both times
results = store.search(namespace, query="preferred name")
matching = [item for item in results if item.key == "preferred_name"]
assert len(matching) == 1 # one row, not two -- the second put overwrote the
assert matching[0].value["text"] == "Danielle"

if __name__ == "__main__":
    test_forgettable_message_is_not_saved()
    print("PASS: a forgettable message writes nothing to the Store")
    test_corrected_name_overwrites_instead_of_duplicating()
    print("PASS: a corrected name overwrites the old value instead of duplicating

```

Expected output:

```

PASS: a forgettable message writes nothing to the Store
PASS: a corrected name overwrites the old value instead of duplicating it

```

Difference from Intermediate, and what this approach adds: Intermediate proves the judge classifies correctly in isolation. Advanced proves the two things that actually matter once this is wired into a real graph: a message judged not worth saving genuinely leaves the Store untouched (not just "the code ran"), and two different phrasings of a correction land on the same key, so `store.put`'s overwrite behavior does the deduping for you -- no similarity search, no embeddings, no second "is this like something I already know?" model call needed.

Which one should you actually write? The fixed-key approach (Intermediate's `ALLOWED_KEYS` plus Advanced's `write_back_node`) is what you should keep. It's simple enough to explain in an interview in one sentence -- "facts are saved under a small set of known keys, so a new phrasing overwrites instead of duplicating" -- and it's genuinely correct for the kind of facts this project cares about (a name, a preference). A more general system, remembering arbitrary open-ended facts with no fixed key list, would need real similarity search over existing memories before writing a new one - - worth knowing that's the next step up, but well past what this project needs.